COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010/DATA STRUCTURES AND ALGORITHM	**SECTION:**	CPE21S1
DATE PERFORMED:	22/9/2026	**DATE SUBMITTED:**	30/9/2026
NAME:	LOREEN JAE D. PILAPIL	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

ACTIVITY NO. 9 **TREE ADT**

1. PROCEDURE OUTPUT

ILO A:

```
int main(){

    GeneralTree<char> tree('A');

    tree.addChild('A', 'B');
    tree.addChild('A', 'C');
    tree.addChild('A', 'D');
    tree.addChild('A', 'E');
    tree.addChild('A', 'F');
    tree.addChild('A', 'G');

    tree.addChild('D', 'H');

    tree.addChild('E', 'I');
    tree.addChild('E', 'J');

    tree.addChild('F', 'K');
    tree.addChild('F', 'L');
    tree.addChild('F', 'M');

    tree.addChild('G', 'N');

    tree.addChild('J', 'P');
    tree.addChild('J', 'Q');
```

```cpp
template <typename T>
class GeneralTree{
    private:
    T key;
    GeneralTree<T>* firstChild;
    GeneralTree<T>* nextSibling;

    public:
    GeneralTree(T value);
    ~GeneralTree();

    GeneralTree<T>* find(T value);
    bool addChild(T parentValue,T childValue);
};

template <typename T>
GeneralTree<T>::GeneralTree(T value){
    key= value;
    firstChild= nullptr;
    nextSibling= nullptr;
}

template <typename T>
GeneralTree<T>::~GeneralTree(){
    delete firstChild;
    delete nextSibling;
}

template <typename T>
GeneralTree<T>* GeneralTree<T>::find(T value){
    if(key == value){
        return this;
    }

    if(firstChild != nullptr){
        GeneralTree<T>* found= firstChild->find(value);
        if(found != nullptr){
            return found;
        }
    }
```

```
    if(nextSibling != nullptr){
        return nextSibling->find(value);
    }
    return nullptr;
}

template <typename T>
bool GeneralTree<T>::addChild(T parentValue,T childValue){
    GeneralTree<T>* parent= find(parentValue);
    if(parent == nullptr){
        return false;
    }

    GeneralTree<T>* newNode = new GeneralTree<T>(childValue);

    if(parent->firstChild == nullptr){
        parent->firstChild= newNode;
    }else{
        GeneralTree<T>* current= parent->firstChild;
        while(current->nextSibling != nullptr){
            current= current->nextSibling;
        }
        current->nextSibling= newNode;
    }
    return true;
}

#endif
```

This created the tree. Each node has a firstChild pointer for its list of children and a nextSibling pointer for the next child of the same parent. This allows a node to have multiple children, such as A, which has six children.

Table 9.1

NODE	HEIGHT	DEPTH
A	3	0
B	0	1
C	0	1
D	1	1
E	2	1
F	1	1
G	1	1

H	0	2
I	0	2
J	1	2
K	0	2
L	0	2
M	0	2
N	0	2
P	0	3
Q	0	3

Table 9.2

ILOB:

PRE-ORDER	**A B C D H E I J P Q F K L M G N**
POST-ORDER	**B A C H D I E P J Q K F L M N G**
IN-ORDER	**B C H D I P Q J E K L M F N G A**

Table 9.3

PRE-ORDER	`template <typename T>`
	`void GeneralTree<T>::preOrder(){`
	`  std::cout<< key << "  ";`
	`  for(GeneralTree<T>* current = firstChild; current != nullptr; current=current->nextSibling){`
	`  current->preOrder();`
	`  }`
	`}`

POST-ORDER	`template <typename T>`
	`void GeneralTree<T>::postOrder(){`
	`  for(GeneralTree<T>* current = firstChild; current != nullptr; current=current->nextSibling){`
	`  current->postOrder();`
	`  }`
	`  std::cout<< key << "  ";`
	`}`

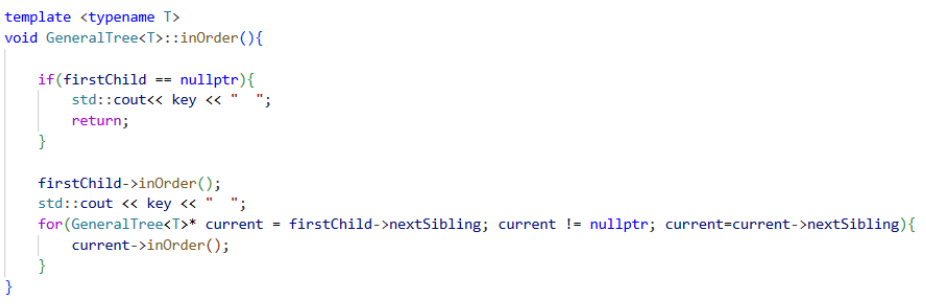
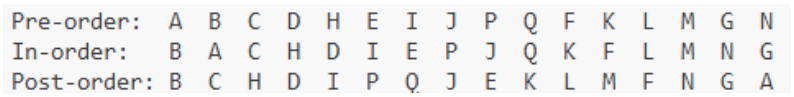
IN-ORDER	
OUTPUT	
OBSERVATION	The output matched the manually answered tree above. All three traversals visit the same nodes, but the position of the parent changes depending on the traversal type.

Table 9.4

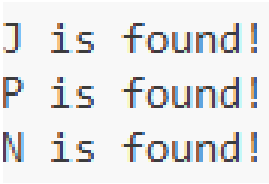


Table 9.5

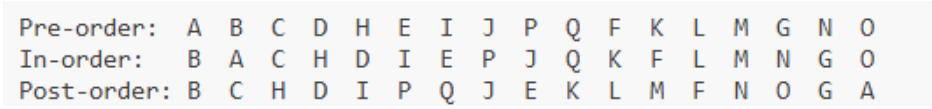
OUTPUT	
ANSWER	The new node changes all three traversal outputs. In pre-order and in-order it is added at the end, and in post-order it is added before letter G.

Table 9.6

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve given problems using the tree data structure's implementation in C++

Step 1: Implement a binary search tree that will take the following values: 2, 3, 9, 18, 0, 1, 4, 5.

```cpp
int main(){

    BinarySearchTree<int> tree(2);

    tree.insert(3);
    tree.insert(9);
    tree.insert(18);
    tree.insert(0);
    tree.insert(1);
    tree.insert(4);
    tree.insert(5);

    return 0;
}
```

```cpp
template <typename T>
class BinarySearchTree{
    private:
    T key;
    BinarySearchTree<T>* left;
    BinarySearchTree<T>* right;

    public:
    BinarySearchTree(T value);
    ~BinarySearchTree();

    void insert(T value);
    void preOrder();
    void inOrder();
    void postOrder();
};

template <typename T>
BinarySearchTree<T>::BinarySearchTree(T value){
    key= value;
    left= nullptr;
    right= nullptr;

}

template <typename T>
BinarySearchTree<T>::~BinarySearchTree(){
    delete left;
    delete right;
}
```

```cpp
template <typename T>
void BinarySearchTree<T>::insert(T value){

    if(value < key){
        if(left == nullptr){
            left =new BinarySearchTree<T>(value);
        }else{
            left->insert(value);
        }
    }
    else if(value>key){
        if(right == nullptr){
            right =new BinarySearchTree<T>(value);
        }else{
            right->insert(value);
        }
    }
}

template <typename T>
void BinarySearchTree<T>::preOrder(){

    std::cout<<key << "  ";
     if(left != nullptr){
        left->preOrder();
     }
     if(right != nullptr){
        right->preOrder();
     }
}

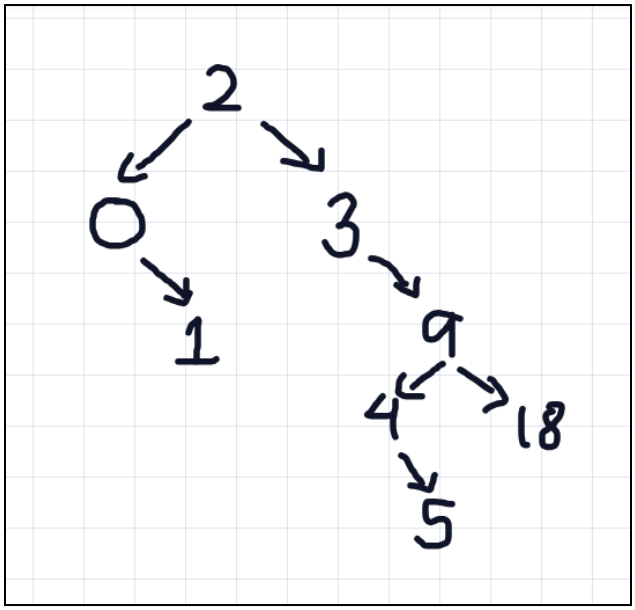
template <typename T>
void BinarySearchTree<T>::inOrder(){
    if(left != nullptr){
        left->inOrder();
    }
    std::cout<<key<< "  ";
    if(right != nullptr){
        right->inOrder();
    }
}
```

```
template <typename T>
void BinarySearchTree<T>::postOrder(){
    if(left != nullptr){
        left->postOrder();
    }
    if(right != nullptr){
        right->postOrder();
    }
    std::cout<<key << "  ";
}

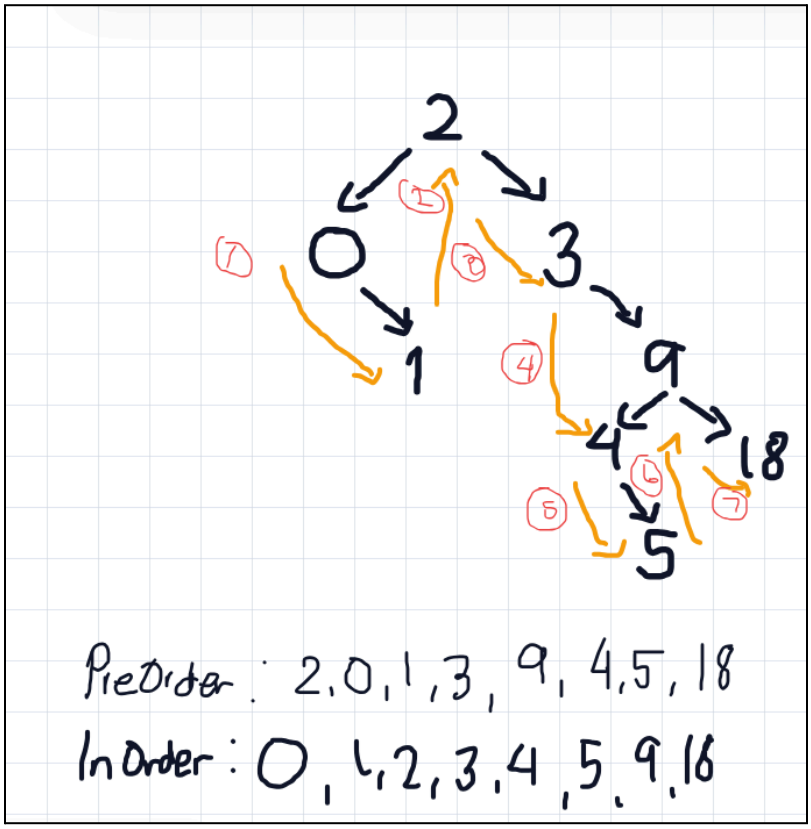
#endif
```

(Screenshot of code)

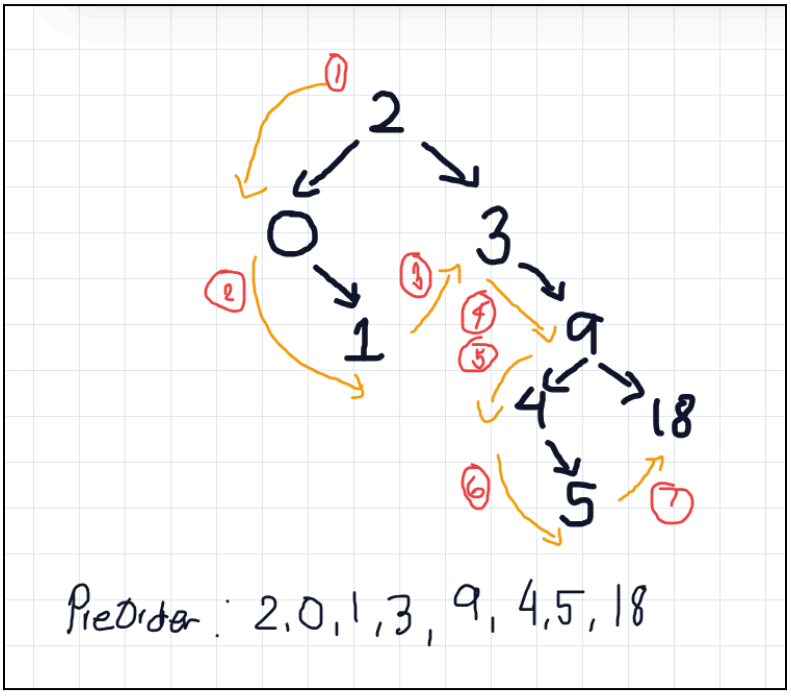
Step 2: Create a diagram to show the tree after all values have been inserted. Then, with the use of visual aids (like arrows and numbers) indicate the traversal order for in-order, pre-order and post-order traversal on the diagram.



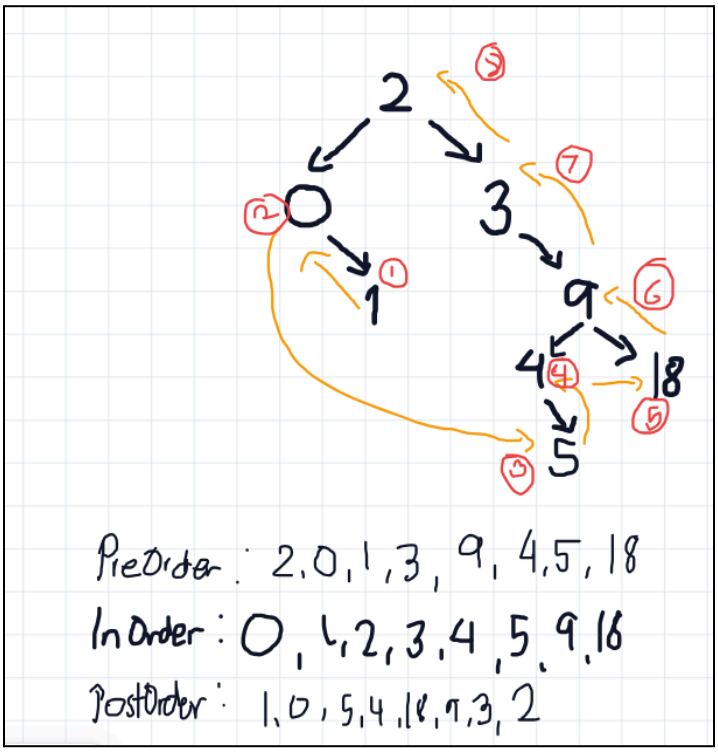
(Screenshot of tree diagram)



(Screenshot of tree diagram with indicated in-order traversal)



(Screenshot of tree diagram with indicated pre-order traversal)



(Screenshot of tree diagram with indicated post-order traversal)

Step 3: Compare the different traversal methods. In-order traversal was performed with what function?

```
Pre-order: 2  0  1  3  9  4  5  18
In-order: 0  1  2  3  4  5  9  18
Post-order: 1  0  5  4  18  9  3  2
```

Inorder traversal was performed with the inOrder() function in the .h file.
(Screenshot output needed)

Given the same input values above, what is the output with different traversal methods? For each output below, indicate your observation: is the output different from the pre-order and post-order traversal that you indicated in the diagrams shown in item #2.

Pre-order Traversal

```
template <typename T>
void BinarySearchTree<T>::preOrder(){

    std::cout<<key << "  ";
    if(left != nullptr){
        left->preOrder();
    }
    if(right != nullptr){
        right->preOrder();
    }
}
```

(Screenshot created function)

```
Pre-order: 2  0  1  3  9  4  5  18
In-order: 0  1  2  3  4  5  9  18
Post-order: 1  0  5  4  18  9  3  2
```

(Screenshot console output)

The root 2 comes first and every parent appears before its children

Post-order Traversal

```
template <typename T>
void BinarySearchTree<T>::postOrder(){
    if(left != nullptr){
        left->postOrder();
    }
    if(right != nullptr){
        right->postOrder();
    }
    std::cout<<key << "  ";
}
```

(Screenshot created function)

```
Pre-order: 2  0  1  3  9  4  5  18
In-order: 0  1  2  3  4  5  9  18
Post-order: 1  0  5  4  18  9  3  2
```

(Screenshot console output)

Every node appears after its children, so the root 2 comes last

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.